

CENTRO UNIVERSITÁRIO DE PATOS DE MINAS – UNIPAM

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

TURMA: 5º PERÍODO **DISCIPLINA:** TÓPICOS INTEGRADORES I **PROFESSOR:** RAFAEL MARINHO E SILVA

VALOR: 02 PONTOS **DATA DE ENTREGA (até o final da aula):** Turma A 22/05/2026 | Turma B: 21/05/2026

ATIVIDADE AVALIATIVA – MongoDB

Cenário: Sistema de Gerenciamento da Loja Virtual "TechStore"

A loja virtual **TechStore** comercializa produtos eletrônicos (notebooks, smartphones, periféricos, acessórios) e precisa de um banco de dados capaz de armazenar **produtos com atributos variáveis** (um notebook tem processador e memória RAM; um cabo HDMI tem apenas comprimento e versão), **clientes com históricos de compras e pedidos com listas de itens aninhadas**.

Por que MongoDB e não um SGBD relacional?

- O catálogo possui produtos com **estruturas heterogêneas** — em um banco relacional seriam necessárias dezenas de tabelas ou colunas nulas.
- Pedidos contêm **listas de itens (arrays embutidos)**, naturalmente representáveis em documentos JSON.
- A operação de e-commerce exige **alta velocidade de leitura e escalabilidade horizontal**, característica nativa do MongoDB.
- O esquema do catálogo **evolui constantemente** (novas categorias, novos atributos), e o MongoDB suporta isso sem migrations rígidas.

Instruções gerais

A atividade deverá ser resolvida **individualmente**, diretamente no laboratório, e **apresentada ao professor ao final da aula** com:

- A janela do **mongosh** aberta, exibindo o histórico de **todos os comandos digitados** pelo aluno.
- A janela do **MongoDB Compass** aberta com o banco **techstore**, para validação visual dos resultados.

O professor passará em cada máquina para validar os resultados.

Regra obrigatória: TODOS os comandos da atividade devem ser digitados manualmente pelo aluno no mongosh. O MongoDB Compass deve ser usado **somente como ferramenta de verificação visual** (conferir documentos, conferir estrutura das coleções, conferir alterações feitas). Não é permitido executar agregações, inserções, atualizações ou remoções diretamente pela interface gráfica do

Compass — o objetivo da prática é que o aluno compreenda e memorize a sintaxe dos comandos.

Para cada questão, o aluno deve:

1. **Digitar o comando** no **mongosh** (terminal).
2. **Conferir o resultado** retornado no terminal.
3. **Validar visualmente** no **MongoDB Compass**, dando *Refresh* na coleção afetada e conferindo o efeito da operação na aba **Documents**.

Ao final da atividade, quando todos os passos estiverem concluídos, o aluno apresentará o resultado final ao professor, conforme descrito na **Questão 16**.

*As ferramentas **mongosh** e **MongoDB Compass** já estão instaladas nos computadores do laboratório. Abra as duas janelas lado a lado: digite os comandos no mongosh e, após cada operação, clique em Refresh na coleção dentro do Compass para visualizar a alteração.*

PARTE I — Configuração e operações básicas (baseada nas Práticas 1 e 2)

Questão 1 — Listando bancos existentes

Execute o comando `show dbs` no mongosh e, em seguida, abra o **MongoDB Compass** e conecte-se ao servidor local (`mongodb://localhost:27017`). Será apresentada a lista de bancos tanto no terminal quanto no painel lateral do Compass.

```
show dbs
```

Questão 2 — Criação do banco de dados

Crie o banco de dados da loja com o comando `use techstore`. Após a primeira inserção (Questão 4), clique em *Refresh* no Compass e o novo banco `techstore` será listado no painel lateral.

```
use techstore
```

Questão 3 — Criação das coleções

Crie as **três coleções** necessárias para o sistema: `clientes`, `produtos` e `pedidos`. Em seguida execute `show collections`:

```
db.createCollection("clientes")
db.createCollection("produtos")
db.createCollection("pedidos")
show collections
```

No Compass, expanda o banco `techstore` e visualize as três coleções criadas no painel lateral.

Questão 4 — Inserindo documentos (`insertMany`)

Insira **no mínimo 4 documentos por coleção**, respeitando a estrutura abaixo. Observe que os produtos têm **campos variáveis por categoria** — isso é justamente uma das principais vantagens do MongoDB.

```
db.clientes.insertMany([
  { nome: "Ana Souza",    idade: 28, cidade: "Patos de Minas", interesses: ["notebook", "smartphone", "acessorio", "monitor"],
  { nome: "Bruno Lima",   idade: 35, cidade: "Uberlândia",    interesses: ["smartphone", "acessorio", "monitor"],
  { nome: "Carla Mendes", idade: 22, cidade: "Patos de Minas", interesses: ["periferico", "acessorio", "monitor"],
  { nome: "Diego Alves",  idade: 41, cidade: "Belo Horizonte", interesses: ["notebook", "smartphone", "acessorio", "monitor"]
])

db.produtos.insertMany([
  { nome: "Notebook Dell Inspiron", categoria: "Notebook",    preco: 4500, estoque: 10,
  { nome: "Smartphone Galaxy S23",  categoria: "Smartphone",  preco: 3200, estoque: 15,
  { nome: "Cabo HDMI 2.1",          categoria: "Acessório",    preco: 60,   estoque: 50,
  { nome: "Monitor LG 27\"",         categoria: "Monitor",      preco: 1800, estoque: 8
])

db.pedidos.insertMany([
  { cliente: "Ana Souza",    itens: [{ produto: "Notebook Dell Inspiron", qtd: 1, preco: 4500 }, { produto: "Smartphone Galaxy S23", qtd: 1, preco: 3200 }, { produto: "Cabo HDMI 2.1", qtd: 2, preco: 60 }, { produto: "Monitor LG 27\"", qtd: 1, preco: 1800 } ],
  { cliente: "Bruno Lima",   itens: [{ produto: "Smartphone Galaxy S23", qtd: 1, preco: 3200 }, { produto: "Cabo HDMI 2.1", qtd: 2, preco: 60 }, { produto: "Monitor LG 27\"", qtd: 1, preco: 1800 } ],
  { cliente: "Carla Mendes", itens: [{ produto: "Cabo HDMI 2.1", qtd: 1, preco: 60 }, { produto: "Monitor LG 27\"", qtd: 1, preco: 1800 } ],
  { cliente: "Diego Alves",  itens: [{ produto: "Monitor LG 27\"", qtd: 2, preco: 1800 } ]
])
```

Em seguida, execute `db.clientes.find()`, `db.produtos.find()` e `db.pedidos.find()` no mongosh. No Compass, abra a aba **Documents** de cada coleção e confira se todos os documentos foram inseridos corretamente.

Questão 5 — `updateOne` substituindo campos

Atualize o cliente **Bruno Lima**, alterando seu nome para **"Bruno Lima Pereira"** e a cidade para **"Araxá"**. Em seguida, execute `db.clientes.find({ nome: "Bruno Lima Pereira" })` e, no Compass, dê *Refresh* na coleção `clientes` e confira o documento atualizado.

```
db.clientes.updateOne(
  { nome: "Bruno Lima" },
  { $set: { nome: "Bruno Lima Pereira", cidade: "Araxá" } }
)
```

Questão 6 — `updateMany` aplicando regra em massa

Aplique um **desconto de 10%** em todos os produtos cujo preço seja maior que **1000**, usando o operador `$mul`. Depois, execute `db.produtos.find({ preco: { $gt: 1000 } })` e, no Compass, use o campo de filtro (Filter) da coleção `produtos` com `{ preco: { $gt: 1000 } }` para visualizar os documentos afetados.

```
db.produtos.updateMany(
  { preco: { $gt: 1000 } },
  { $mul: { preco: 0.9 } }
)
```

Questão 7 — Manipulando arrays (`$push` e `$each`)

Adicione novos interesses ao array `interesses` da cliente **Ana Souza**, simulando o comportamento de recomendação por categoria visitada:

```
db.clientes.updateOne(
  { nome: "Ana Souza" },
  { $push: { interesses: { $each: ["monitor", "teclado mecânico"] } } }
)
```

No Compass, abra o documento da cliente Ana Souza e confira o array `interesses` expandido com os novos valores.

Questão 8 — Atualização condicional (`$inc`)

Após a venda de um cabo HDMI, diminua o estoque desse produto em **1 unidade** usando `$inc` com valor negativo. Em seguida, valide com `findOne`:

```
db.produtos.updateOne(
  { nome: "Cabo HDMI 2.1" },
  { $inc: { estoque: -1 } }
)
db.produtos.findOne({ nome: "Cabo HDMI 2.1" })
```

No Compass, confirme que o campo `estoque` do `Cabo HDMI 2.1` passou de 80 para 79.

Questão 9 — Removendo documentos

O pedido da cliente **Carla Mendes** foi cancelado (status `pendente`). Remova esse pedido com `deleteOne`:

```
db.pedidos.deleteOne({ cliente: "Carla Mendes", status: "pendente" })
db.pedidos.find()
```

No Compass, dê *Refresh* na coleção `pedidos` e confira que o pedido da Carla Mendes não consta mais.

PARTE II — Funções e cálculos com `aggregate` (baseada nas Práticas 3 e 4)

*Esta parte aborda o conteúdo que **não foi cobrado** na lista de exercícios original: a criação de **funções JavaScript no mongosh** que encapsulam operações de CRUD e cálculos com o framework de agregação.*

Questão 10 — Função de cadastro

Crie uma função `cadastrarProduto` que receba a categoria e os demais atributos, e insira o documento na coleção `produtos`. Em seguida, utilize-a para cadastrar **dois novos produtos**:

```
const cadastrarProduto = (nome, categoria, preco, estoque, atributosExtras = {}) => {
  db.produtos.insertOne({ nome, categoria, preco, estoque, ...atributosExtras });
  print(`Produto "${nome}" cadastrado na categoria ${categoria}.`);
};

cadastrarProduto("Mouse Logitech MX", "Acessório", 350, 25, { dpi: 4000 });
cadastrarProduto("Teclado Mecânico K2", "Acessório", 480, 12, { switch: "Brown" });
```

Atualize a coleção **produtos** no Compass e confira os dois novos documentos com seus atributos extras (**dpi** e **switch**) visíveis.

Questão 11 — Função de leitura por filtro

Crie a função **listarPorCategoria**, que recebe uma categoria e imprime todos os produtos pertencentes a ela.

```
const listarPorCategoria = (categoria) => {
  const docs = db.produtos.find({ categoria }).toArray();
  print(`--- Produtos da categoria ${categoria} (${docs.length}) ---`);
  docs.forEach(d => printjson(d));
};

listarPorCategoria("Acessório");
```

Questão 12 — Função de edição

Crie a função **editarPreco**, que altera o preço de um produto pelo nome.

```
const editarPreco = (nome, novoPreco) => {
  const r = db.produtos.updateOne({ nome }, { $set: { preco: novoPreco } });
  if (r.modifiedCount > 0) print(`Preço de "${nome}" atualizado para ${novoPreco}.`);
  else print(`Produto "${nome}" não encontrado.`);
};

editarPreco("Mouse Logitech MX", 320);
```

Questão 13 — Função de cálculo: média de preços (\$avg)

Digite no mongosh uma função que calcule a **média de preço por categoria** usando **aggregate** com **\$group** e **\$avg**. Confira o **resultado impresso no terminal do mongosh** após a execução da função. Em seguida, no Compass, abra a coleção **produtos** (aba *Documents*) e confira manualmente se as médias calculadas correspondem aos preços dos produtos cadastrados.

```
const mediaPrecoPorCategoria = () => {
  const resultado = db.produtos.aggregate([
    { $group: { _id: "$categoria", mediaPreco: { $avg: "$preco" }, totalProdutos: { $sum: 1 } } },
    { $sort: { mediaPreco: -1 } }
  ]).toArray();
  print("==== Média de preço por categoria =====");
  resultado.forEach(r => print(`${r._id}: R$ ${r.mediaPreco.toFixed(2)} ($${r.totalProdutos})`);
};

mediaPrecoPorCategoria();
```

Questão 14 — Função de cálculo: faturamento total por cliente (\$sum, \$unwind)

Digite no mongosh uma função que calcule o **faturamento total** de cada cliente, considerando todos os itens dos pedidos pagos. Esta função usa três estágios do pipeline de agregação: **\$match**, **\$unwind** e **\$group**. Confira o **resultado impresso no terminal do mongosh** e, no Compass, abra a coleção **pedidos** (aba *Documents*) para conferir manualmente se os totais por cliente batem com os pedidos cadastrados.

```
const faturamentoPorCliente = () => {
  const resultado = db.pedidos.aggregate([
    { $match: { status: "pago" } },
    { $unwind: "$itens" },
    { $group: {
      _id: "$cliente",
      totalGasto: { $sum: { $multiply: ["$itens.qtd", "$itens.preco"] } }
    } },
    { $sort: { totalGasto: -1 } }
  ]).toArray();
  print("==== Faturamento por cliente =====");
  resultado.forEach(r => print(`${r._id}: R$ ${r.totalGasto.toFixed(2)}`));
};

faturamentoPorCliente();
```

Questão 15 — Função composta: registrar venda (transação simulada)

Inspirada na função de **transferência PIX** da Prática 4, crie a função **registrarVenda** que, em uma única chamada:

1. Verifica se o produto possui estoque suficiente.
2. Decrementa o estoque do produto (**\$inc**).
3. Insere um novo pedido na coleção **pedidos**.
4. Imprime o resultado da operação.

```
const registrarVenda = (clienteNome, produtoNome, quantidade) => {
  const produto = db.produtos.findOne({ nome: produtoNome });
  if (!produto) { print(`Produto "${produtoNome}" não encontrado.`); }
  if (produto.estoque < quantidade) {
    print(`Estoque insuficiente. Disponível: ${produto.estoque} | Pedido: ${quantidade}`);
    return;
  }
  db.produtos.updateOne({ nome: produtoNome }, { $inc: { estoque: -quantidade } });
  db.pedidos.insertOne({
    cliente: clienteNome,
    itens: [{ produto: produtoNome, qtd: quantidade, preco: produto.preco }],
    status: "pago",
    data: new Date()
  });
  print(`Venda registrada: ${quantidade}x "${produtoNome}" para ${clienteNome}. Total: R$ ${produto.preco * quantidade}`);
};

registrarVenda("Ana Souza", "Teclado Mecânico K2", 1);
registrarVenda("Diego Alves", "Mouse Logitech MX", 2);
registrarVenda("Bruno Lima Pereira", "Notebook Dell Inspiron", 999); // Deve falhar
```

Verifique, no Compass:

- O estoque do **Teclado Mecânico K2** reduzido de 12 para 11.
- O estoque do **Mouse Logitech MX** reduzido de 25 para 23.
- Os dois novos pedidos visíveis na coleção **pedidos**.
- E, no mongosh, a mensagem de erro de estoque na tentativa de venda do Notebook (saída da função).

PARTE III — Apresentação final ao professor

Questão 16 — Apresentação dos resultados

Esta é a **única etapa** em que o aluno **não digita comandos** — serve para apresentar ao professor o estado final do banco, juntamente com o histórico de comandos digitados no mongosh.

Quando o professor passar em sua mesa para avaliar a atividade, apresente:

No mongosh:

- O histórico completo de todos os comandos digitados ao longo da atividade.
- As saídas das funções criadas nas Questões 13, 14 e 15 (médias por categoria, faturamento por cliente e registros de venda).

No **MongoDB Compass**, abra o banco **techstore** e apresente, para cada uma das três coleções (**clientes**, **produtos**, **pedidos**):

- O total de documentos exibido no painel da coleção.
- Os documentos visíveis na aba **Documents**, evidenciando que o conteúdo corresponde ao que foi inserido, alterado e removido durante a atividade pelos comandos do mongosh.

Boa atividade!